Lab 7: Fuzzing for Input Validation Bugs

Blood Donation Screening (processor.py) — Hypothesis property-based testing

Name	Uditi Mourya	**SRN**	PES2UG24CS565
Course	B.Tech CSE, PES University	**Lab**	Lab 7 — Fuzz Testing
Repo	github.com/uditiii/PES2UG24CS565_L6_L7_05_blood_donation_screening		

1. Aim

Use Hypothesis to fuzz three input-handling functions of a blood-donation screening module, find the inputs that crash them, and harden the code so none of them can raise on any input.

2. What fuzzing and Hypothesis do here

Fuzzing feeds random or malformed input to a program and watches for crashes. Hypothesis does this as property-based testing: the property I assert is **"this function never raises, whatever string or None it is given"**. Hypothesis generates many inputs from `st.text() | st.none()`, and when one fails it shrinks it to the smallest failing case (e.g. `''`, `'|'`, `None`).

3. Setup (Step 1)

```
$ pip install hypothesis pytest

Python 3.13, hypothesis 6.170.14, pytest 9.1.1
```

4. Step 2 — running the buggy processor.py

I gave it a valid blood type and then a diastolic of 0. The first function is fine; the second crashes the whole program with an unhandled `ZeroDivisionError`.

```
$ python processor.py

==== Running validate_blood_type ====
Enter blood type (e.g. A+, O-, AB+): Validated: AB Positive

==== Running calculate_pulse_pressure ====
Enter BP (e.g. 120/80): Traceback (most recent call last):
  File "/home/uditi/Lab7/processor.py", line 50, in <module>
    main()
    ~~~~^^
  File "/home/uditi/Lab7/processor.py", line 43, in main
    print("BP Metrics:", calculate_pulse_pressure(bp_input))
                        ~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "/home/uditi/Lab7/processor.py", line 21, in calculate_pulse_pressure
    ratio = round(systolic / diastolic, 2)
                  ~~~~~~~~~^~~~~~~~~~
ZeroDivisionError: division by zero
```

5. Step 3 — completed test_processor.py

Each test wraps the call in try/except and fails with the exception type and message if anything is raised.

```
from hypothesis import given, strategies as st
from processor import validate_blood_type, calculate_pulse_pressure, parse_donor_registry


@given(st.text() | st.none())
def test_validate_blood_type_no_crash(s):
    try:
        validate_blood_type(s)
    except Exception as e:
        assert False, f"validate_blood_type crashed with: {type(e).__name__}: {e}"


@given(st.text() | st.none())
def test_calculate_pulse_pressure_safe(s):
    try:
```

```
        calculate_pulse_pressure(s)
    except Exception as e:
        assert False, f"calculate_pulse_pressure crashed with: {type(e).__name__}: {e}"


@given(st.text() | st.none())
def test_parse_donor_registry_safe(s):
    try:
        parse_donor_registry(s)
    except Exception as e:
        assert False, f"parse_donor_registry crashed with: {type(e).__name__}: {e}"
```

6. Step 4 — pytest against the buggy code (all 3 fail)

Hypothesis found 3 distinct failures in every function (9 total). Below is the pytest output trimmed to the essentials: the test header, how many distinct failures Hypothesis found per test, the minimised failing input (`s=...`) and the exception it caused, and the final summary.

```
$ python -m pytest test_processor.py

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/uditi/Lab7
plugins: platformdirs-4.13.5, anyio-4.15.1, hypothesis-6.170.14
collected 3 items

test_processor.py FFF                                                    [100%]

=================================== FAILURES ===================================
_____________________ test_validate_blood_type_no_crash ______________________
   ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
     AssertionError: validate_blood_type crashed with: KeyError: '0'
     Failing test case: test_validate_blood_type_no_crash(
         s='0',
     AssertionError: validate_blood_type crashed with: IndexError: string index out of range
     Failing test case: test_validate_blood_type_no_crash(
         s='',
     AssertionError: validate_blood_type crashed with: AttributeError: 'NoneType' object has no attribute 'strip'
     Failing test case: test_validate_blood_type_no_crash(
         s=None,
_____________________ test_calculate_pulse_pressure_safe _____________________
   ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
     AssertionError: calculate_pulse_pressure crashed with: ValueError: invalid literal for int() with base 10: ''
     Failing test case: test_calculate_pulse_pressure_safe(
         s='/',
     AssertionError: calculate_pulse_pressure crashed with: ValueError: not enough values to unpack (expected 2, got
1)
     Failing test case: test_calculate_pulse_pressure_safe(
         s='',  # or any other generated value
     AssertionError: calculate_pulse_pressure crashed with: AttributeError: 'NoneType' object has no attribute 'split'
     Failing test case: test_calculate_pulse_pressure_safe(
         s=None,  # or any other generated value
_____________________ test_parse_donor_registry_safe ______________________
   ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
     AssertionError: parse_donor_registry crashed with: ValueError: could not convert string to float: ''
     Failing test case: test_parse_donor_registry_safe(
         s='|',
     AssertionError: parse_donor_registry crashed with: IndexError: list index out of range
     Failing test case: test_parse_donor_registry_safe(
         s='',  # or any other generated value
     AssertionError: parse_donor_registry crashed with: AttributeError: 'NoneType' object has no attribute 'split'
     Failing test case: test_parse_donor_registry_safe(
         s=None,  # or any other generated value
=========================== short test summary info ============================
FAILED test_processor.py::test_validate_blood_type_no_crash - ExceptionGroup:...
FAILED test_processor.py::test_calculate_pulse_pressure_safe - ExceptionGroup...
FAILED test_processor.py::test_parse_donor_registry_safe - ExceptionGroup: Hy...
============================== 3 failed in 1.54s ===============================
```

7. Bugs found

Function	Input	Exception	Cause
validate_blood_type	`None`	AttributeError	.strip() on non-string
validate_blood_type	`''`	IndexError	cleaned[-1] on empty string
validate_blood_type	`'0'`	KeyError	Rh char not in +/- map; antigen never checked
calculate_pulse_pressure	`None`	AttributeError	.split() on non-string

Function	Input	Exception	Cause
calculate_pulse_pressure	`'0'`	ValueError	no "/" so unpacking fails
calculate_pulse_pressure	`'/'`	ValueError	int('') on empty part
calculate_pulse_pressure	`'120/0' (Step 2)`	ZeroDivisionError	diastolic of 0
parse_donor_registry	`None`	AttributeError	.split() on non-string
parse_donor_registry	`''`	IndexError	fields[1] missing
parse_donor_registry	`'|'`	ValueError	float('') on empty weight

Beyond what Hypothesis surfaced, I also hardened cases it will not find by chance: `float("nan")`/`"inf"` for weight and haemoglobin (which silently give a wrong eligibility result), negative or zero values, wrong token counts (a | b | c | d), and an unknown blood group such as `C+` (the original accepted any letters).

8. Step 5 — the fix

Every function now type-checks its input, validates the format, catches conversion errors, and returns a clear *Invalid ...* message instead of raising. Rules: blood type must be A, B, AB or O followed by + or -; BP must be two positive integers with systolic ≥ diastolic and diastolic > 0; donor record must have exactly 3 pipe-separated fields, a non-empty ID, and finite positive weight and haemoglobin.

```
import math

VALID_ABO = {"A", "B", "AB", "O"}
RH_MAP = {"+": "Positive", "-": "Negative"}


def validate_blood_type(blood_code):
    """
    Standardizes blood group and Rh factor (e.g. 'O+', 'AB-').
    Returns 'Invalid blood type' for anything that is not a valid ABO group
    followed by a +/- Rh sign (None, empty, non-string, unknown group, etc.).
    """
    if not isinstance(blood_code, str):
        return "Invalid blood type"
    cleaned = blood_code.strip().upper()
    if len(cleaned) < 2:
        return "Invalid blood type"
    antigen, rh_char = cleaned[:-1], cleaned[-1]
    if antigen not in VALID_ABO or rh_char not in RH_MAP:
        return "Invalid blood type"
    return f"{antigen} {RH_MAP[rh_char]}"


def calculate_pulse_pressure(bp_string):
    """
    Calculates pulse pressure (systolic - diastolic) and arterial ratio.
    Expects 'systolic/diastolic' (e.g. '120/80') with positive integers and
    systolic >= diastolic. Returns 'Invalid BP reading' otherwise.
    """
    if not isinstance(bp_string, str):
        return "Invalid BP reading"
    parts = bp_string.split('/')
    if len(parts) != 2:
        return "Invalid BP reading"
    try:
        systolic = int(parts[0])
        diastolic = int(parts[1])
    except ValueError:
        return "Invalid BP reading"
    if diastolic <= 0 or systolic < diastolic:
        return "Invalid BP reading"
    pulse_diff = systolic - diastolic
    try:
        ratio = round(systolic / diastolic, 2)
    except (OverflowError, ZeroDivisionError):
        return "Invalid BP reading"
    return f"Diff: {pulse_diff}, Ratio: {ratio}"


def parse_donor_registry(donor_entry):
    """
    Parses pipe-delimited record: 'DonorID|WeightKg|Hemoglobin'.
    Returns 'Invalid donor record' for non-strings, wrong token count, empty
    ID, non-numeric / non-finite / non-positive weight or hemoglobin.
```

```
    """
    if not isinstance(donor_entry, str):
        return "Invalid donor record"
    fields = donor_entry.split('|')
    if len(fields) != 3:
        return "Invalid donor record"
    donor_id = fields[0].strip()
    if not donor_id:
        return "Invalid donor record"
    try:
        weight = float(fields[1])
        hemoglobin = float(fields[2])
    except ValueError:
        return "Invalid donor record"
    if not (math.isfinite(weight) and math.isfinite(hemoglobin)):
        return "Invalid donor record"
    if weight <= 0 or hemoglobin <= 0:
        return "Invalid donor record"
    eligible = weight >= 50.0 and hemoglobin >= 12.5
    return f"Donor {donor_id}: {'Eligible' if eligible else 'Deferred'}"
```

Same inputs as Step 2 (plus a valid donor record). The program now finishes instead of crashing:

```
$ python processor.py

==== Running validate_blood_type ====
Enter blood type (e.g. A+, O-, AB+): Validated: AB Positive

==== Running calculate_pulse_pressure ====
Enter BP (e.g. 120/80): BP Metrics: Invalid BP reading

==== Running parse_donor_registry ====
Enter record (ID|Weight|Hemoglobin): Status: Donor D101: Eligible
```

9. Step 6 — pytest against the fixed code

```
$ python -m pytest test_processor.py

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/uditi/Lab7
plugins: platformdirs-4.13.5, anyio-4.15.1, hypothesis-6.170.14
collected 3 items

test_processor.py ...                                                    [100%]

============================== 3 passed in 0.29s ===============================
```

I also re-ran each test with 20,000 examples (instead of the default 100) and all three still passed, so the fix is not just lucky on a small sample.

10. Conclusion

Fuzzing found nine crashing inputs in about a second, all of them cases the original docstrings said were "assumed" not to happen. The fix is defensive validation at the start of each function. The trade-off is that invalid input now returns an error string rather than raising, so a caller must check the return value; for a real system I would raise a custom exception or return a result object instead.

Note: terminal blocks are the captured output of each command (Step 4 trimmed to the key lines).